

28.10.2025 - Weekly Meeting

Freitag, 17. Oktober 2025 22:08

Beschlüsse letzte Woche

- Anfangen die Algorithmen von michi einpflegen in Simulationen
- Wunsch: vergleich zwischen Messung und Simulation
 - o Simulation sollte passen wie Faust aufs Auge

Python Portierung:

- Matlab code nicht weiterentwickeln
- wenn in python etwas anders gemacht wird -> auch in MATLAB zurückerpflegen
- gibt schon ein file bei michi in GitHub
PES_Board/docs/Solutions/python/serial_stream_pi_controller.jpytr

Aufgaben seit letzter Woche:

Yuri:

- RPM Filter und drone tunen mit 6S akku
- Codestruktur MATLAB

Dario:

- Header zeugs
- Entscheid nächste Woche ob weiterverfolgen oder nicht
- Python schauen was für librarys

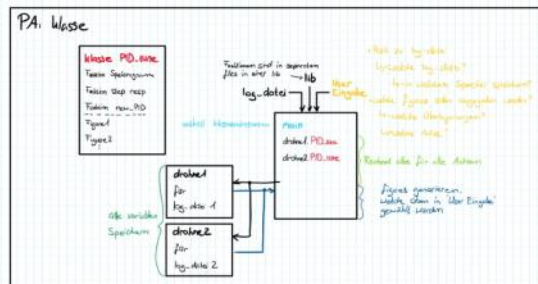
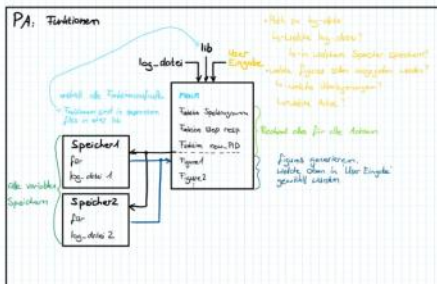
Janick:

- Simulationen

Erladigte Aufgaben

16. Okt	Janick	Mit Anpassungen Regler begonnen. Code neu strukturiert, Daten für Sprungantwort importiert
16. Okt	Yuri, Dario, Janick	Drohnenflug Skills aufgebessert (Flugnachmittag)
16. Okt	Yuri	Begonnen mit ordnerstruktur für strukturierter MATLAB code
17. Okt	Janick	Fertigstellung der Spektogrammsimulation
17. Okt	Janick	Import der Funktion aus der Library für einen Vergleich
20. Okt	Yuri	Codestruktur angepasst und Ordnerstruktur danach in MATLAB erstellt, Packages -> einfacher für Python Portierung begonnen mit Aufteilung
21. Okt	Janick Yuri	Simulation Plant mit Funktion von Michi abgeglichen (Erfolgreich :)) Verucht RPM filter zum laufen zu bringen -> ohne Erfolg... zu alte Firmware und upgrade nicht direkt möglich
21. Okt	Janick	Simulation Spectrum mit Funktion von Michi abgeglichen (Erfolgreich :))
21. Okt	Janick	Simulation Regler mit Funktion von Michi abgeglichen (nicht Erfolgreich :()
21. Okt	Dario	Identisches header file generiert mit hilfe des blackbox log viewer, Schritt Antwort hat immer noch Schwingungen ☹️, sollte möglich sein jedoch unklar welche Werte noch skaliert werden müssen
23. Okt	Yuri	Frequency_response klasse geschrieben und im main eingefügt
23. Okt	Yuri/ Janick	Besprechung wie die Klassen strukturiert werden und wie der Code Aufbau aussehen soll
23. Okt	Dario	Im Source Code von blackbox-log-viewer herumgestöbert um scaling faktoren zu finden, Unterschiede: loopiteratino, time, rxSignal, heading, Flags
24. Okt	Janick	Erstellung des vorläufigen Main und Main_class Teil
26. Okt	Janick	Vorbereitung Weekly Meeting
27. Okt	Janick	Weiterarbeit Klassen und weitere Test dazu
28. Okt	Janick	Herausziehen der Plots, einlesen in möglichkeiten der Codestruktur und auflistung der möglichen Darstellung der Plots mit mehreren Flugdaten
28. Okt	Dario	Blackbox_decode flags angepasst, ANGLE_MODE nun identisch, leider immer noch schwingungen, Verdacht: loopiteration und time sind unterschiedlich wobei das blackbox_tool ein anderes sambling betreibt und so die Daten nicht gleicht manipuliert werden können

Umbau Matlab Code



Main

- Eingabe der Parameter
- Eingabe des Filepath
- Eingabe der bevorzugten Einstellungen

Main_class

- Kopf hinter allen Berechnungen
- Aufrufen der anderen Klassen
- Sammlung der verschiedenen Flugdaten

1. data_io

- Einlesen der .csv oder .mat-Dateien
- Extraktion von Header-Informationen
- Vorverarbeitung der Daten (Skalierung, Zeitberechnung, etc.)
- Laden/Speichern von Daten

Main

- Eingabe der Parameter
- Eingabe des `filepath`
- Eingabe der bevorzugten Einstellungen

Main class

- Kopf hinter allen Berechnungen
- Aufrufen der anderen Klassen
- Sammlung der verschiedenen Flugdaten

1. data_io

- Einlesen der .csv oder .mat-Dateien
- Extraktion von Header-Informationen
- Vorverarbeitung der Daten (Skalierung, Zeitberechnung, etc.)
- Laden/Speichern von Daten

2. flight_parameters

- Verwaltung von `para`, `para_new`, `para_used`, `ind`, etc.
- Auswahl des Quads und Achse
- Berechnung und Skalierung von PID-Parametern
- Ausgabe der verwendeten Parameter

3. spectrogram

- Berechnung von Spektrogrammen
- Ausgabe von Parametern (z.B. `Nres`, `c_lim`)
- Visualisierung (z.B. `pcolor`, `log scale`, `colormap`)
- Vergleich gefiltert vs. ungefiltert

4. spectra

- Berechnung von Spektren (FFT, Fensterung, `Overlap`)
- Ausgabe von Frequenzbereichen
- Visualisierung der Magnituden (logarithmisch)
- Auswahl der relevanten Datenkanäle

5. frequency_response

- Berechnung von Übertragungsfunktionen (`T`, `Guav`, `Gsw`, `P`)
- Anwendung von Filtern (z.B. `apply_rotfiltfilt`)
- Visualisierung: Bode-Plots, `Coherence`
- Vergleich analytisch vs. gemessen

6. controller_analysis

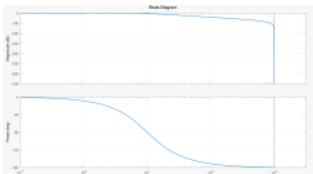
- Berechnung von `Cpl`, `Cd`, `CLana`, `CLana_new`
- Closed-loop Analyse (Tracking, `Sensitivity`, Effort, Compliance)
- Step-Response Berechnung und Darstellung
- Ausgabe der neuen PID-Werte

7. plot_utils

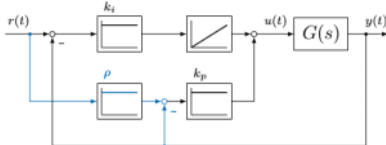
- Gemeinsame Plot-Einstellungen (Farben, Linienbreite, Achsen)
- Funktionen wie `linkaxes`, `expand_multiple_figure_nr`
- Standardisierte Achsenlayouts

Offene Fragen

- Bei der `estimate_spectrum` funktion hat es eine smooth matrix drin. Wie genau funktioniert diese?
- Bei der Funktion `apply_rotfiltfilt` muss man noch eine Übertragungsfunktion `Glp` übergeben. Diese wird dann aber auskommentiert. Für was wäre diese Eigenschaft vorgesehen? Einfach zum hohe Frequenzen rauszufiltern?



- Irgendwas stimmt bei der Berechnung der Übertragungsfunktion im Vergleich zu Simulink nicht. System wird nach Berechnung Instabil aber in Simulink ist es Stabil
- Item Relax, prinzip wie in der Schule gehabt?



Generell Aufgabenteilung:

- Wechsel Arbeiten Dario zu Firmware?

Weiter Schritte:

Janick:

- Weiterführung Matlab Code

Yuri:

- Weiterführung Matlab Code

Dario:

- Arbeit bis jetzt sauber abschliessen
- Csv -> einlesen -> `G_cl()` herleiten -> sprungantwort -> plotten
- https://github.com/pichim/PES_Board/blob/main/docs/solutions/python/serial_stream_pi_controller.ipynb
- https://github.com/pichim/Filter_Implementation_Test/blob/main/docs/python/eval_filter_implementation.ipynb